



**POLITECHNIKA
BYDGOSKA**

Wydział Telekomunikacji,
Informatyki i Elektrotechniki

PRACA DYPLOMOWA INŻYNIERSKA na kierunku Informatyka Stosowana

Analiza obrazów MRI w celu wykrywania zmian

MRI images analysis in order to detect lesions

Zuzanna Tarazewicz

116954

Dr inż. Agata Giełczyk

Bydgoszcz, miesiąc rok

Metryka pracy dyplomowej

Dane ogólne

Nazwa Uczelni	Politechnika Bydgoska im. Jana i Jędrzeja Śniadeckich
Wydział	Telekomunikacji, Informatyki i Elektrotechniki
Kierunek	Informatyka Stosowana
Tryb studiów	stacjonarne
Dane autora	Zuzanna Tarazewicz, 116954
Dane promotora	Dr inż. Agata Giełczyk

Dane dotyczące pracy dyplomowej

Język pracy	język polski [PL]
Tytuł pracy	Analiza obrazów MRI w celu wykrywania zmian
Opis pracy	1. Przegląd literatury tematu dotyczący analizy zdjęć MRI mózgu, przetwarzania obrazów i uczenia maszynowego. 2. Przegląd dostępnych zbiorów danych zawierających MRI mózgu i wybór zbioru danych do uczenia modeli. 3. Przegląd i ocena technik sztucznej inteligencji pod kątem ich przydatności do analizy zdjęć MRI mózgu. Wytypowanie modeli do predykcji. 4. Przygotowanie środowiska do realizacji projektu inżynierskiego online lub lokalnie wraz z repozytorium kodu źródłowego. 5. Implementacja wybranych modeli z użyciem języka Python. Wykorzystanie różnych metod przetwarzania obrazów (np. normalizacja czy rozmycie obrazu) oraz technik uczenia maszynowego (np. dropout czy augmentacja) w celu jak najlepszej identyfikacji zmian chorobowych. 6. Ewaluacja osiągniętych wyników za pomocą metryk ewaluacyjnych (np. F1-score) i wnioski. Celem pracy dyplomowej jest propozycja i implementacja systemu wizji komputerowej służącego do wykrywania zmian chorobowych w mózgu na podstawie zdjęć MRI.
Typ pracy	inżynierska
Streszczenie	Tekst streszczenia w języku polskim. Streszczenie pracy dyplomowej (max. pó ³ strony) powinno zawierać omówienie zagadnień poruszanych w pracy. W części tej należy pokrótce scharakteryzować cel oraz podstawowe założenia pracy.
Słowa kluczowe	sztuczna inteligencja, python, uczenie maszynowe, przetwarzanie obrazów

Diploma thesis record

General information

University name	Bydgoszcz University of Science and Technology
Faculty	Telecommunications, Computer Science and Electrical Engineering
Field of study	Applied Computer Science
Mode of study	full-time
Author's information	Zuzanna Tarazewicz, 116954
Supervisor's information	Dr inż. Agata Giełczyk

Data regarding the diploma thesis

Language of thesis	Polish [PL]
Title of thesis	MRI images analysis in order to detect lesions
Description	Copy from the APD system if the original language is English. The description must be translated into English if the original language is Polish.
Type of thesis	Engineer's
Abstract	The abstract should be written in English and provide a concise summary of the thesis (maximum half a page). It should briefly describe the purpose and basic assumptions of the thesis.
Keywords	artificial intelligence, python, machine learning, image processing

Spis treści

Spis treści	4
1 Wstęp	6
1.1 Motywacje	6
1.2 Cel i zakres pracy	7
1.3 Wyzwania związane z tematem pracy	8
2 Wstęp teoretyczny	10
2.1 Język programowania	10
2.2 Środowisko	11
2.3 Biblioteki	11
2.3.1 Analiza danych	11
2.3.2 Modele uczenia maszynowego	11
2.3.3 Graficzny interfejs użytkownika	12
2.4 Odwołania do literatury	13
2.4.1 Artykuły naukowe	13
3 Prezentacja i omówienie wyników pracy	16
3.1 Klasyfikacja	16
3.1.1 Eksploracyjna analiza danych	16
3.1.2 Trening modeli	18
3.1.3 Porównanie	23
3.2 Detekcja	24
3.2.1 Eksploracyjna analiza danych	24
3.2.2 Trening modelu YOLOv8	25
3.3 Graficzny Interfejs Użytkownika	25

3.4	Działanie modeli w praktyce	26
3.4.1	Grad-CAM	26
3.4.2	Ramki OpenCV	29
3.4.3	YOLOv8	32
4	Wnioski	33
	Bibliografia	34
	Spis rysunków	38
	Spis tabel	39
	Załączniki	40

Rozdział 1

Wstęp

1.1 Motywacje

W dzisiejszych czasach sektor medyczny stoi w obliczu wyzwań związanych z rosnącym zapotrzebowaniem na pomoc medyczną, przy jednoczesnym niedoborze odpowiednio wykształconego personelu. Problem ten dodatkowo nasilił się podczas pandemii wirusa COVID-19 [28], zwiększając obciążenie systemów ochrony zdrowia, wpływając również na psychiczne i fizyczne zdrowie pracowników medycznych. Jednym ze sposobów, którym można przyczynić się do poprawy tej sytuacji jest wprowadzenie zaawansowanych technologii opartych o uczenie maszynowe, które mogłyby wspierać personel medyczny w codziennej pracy. Dzięki możliwościom, które oferuje sztuczna inteligencja, można usprawnić, obecnie często powolne, procesy diagnostyczne. Dzięki analizie danych medycznych w sposób bardziej kompleksowy, szybki a także wspierać podejmowanie decyzji dotyczących odpowiedniego toku leczenia pacjentów.

Obecnie uczenie maszynowe znajduje zastosowanie w wielu dziedzinach codziennego życia, a użycie jego potencjału w medycynie jest ogromny. Dzięki użyciu algorytmów sztucznej inteligencji do wsparcia diagnostyki obrazowej np. zdjęć rezonansu magnetycznego, można wykryć bardzo subtelne zmiany, które mogą umknąć ludzkiemu oku. Pozwoli to na odpowiednią personalizację leczenia poprzez dostosowanie terapii do potrzeb pacjenta.

Jedną z głównych obiekcji w sprawie używania algorytmów sztucznej inteligencji w medycynie jest kwestia odpowiedzialności za ewentualne błędy diagnostyczne. Zrozumiałe jest, że decyzje związane z życiem i zdrowiem ludzi są wyjątkowo wrażliwe i wymagają wyższego poziomu dokładności i precyzji. W tym kontekście niezwykle istotną rolę odrywa AI Act, które jest pierwszym kompleksowym europejskim rozporządzeniem regulującym rozwój i stosowanie sztucznej inteligencji [20]. Zgodnie z jego zasadami systemy sztucznej inteligencji wykorzy-

stywane w ochronie zdrowia są klasyfikowane jako systemy wysokiego ryzyka, co oznacza, że muszą spełniać rygorystyczne wymogi dotyczące nadzoru jakości danych, przejrzystości działania oraz bezpieczeństwa pacjentów. AI Act wprowadza również obowiązek dokumentowania procesu decyzyjnego modeli, regularnych audytów oraz ścisłych testów klinicznych, co ma zwiększyć zaufanie społeczeństwa do tej technologii i znacznie ograniczyć ryzyko błędów [4].

Zamiast postrzegać sztuczną inteligencję jako zagrożenie, powinna ona być traktowana, jako narzędzie do wsparcia lekarzy, a nie jako ich zastępstwo. Poprawa przejrzystości algorytmów, zgodność z wymogami prawnymi, a także edukacja w tym zakresie pomogą rozwiązać obawy związane z wykorzystaniem technologii tego typu w diagnostyce i znacząco zwiększyć akceptację społeczną.

Dzięki ciągłemu rozwojowi sztucznej inteligencji możliwe jest tworzenie coraz bardziej dokładnych, skutecznych i precyzyjnych narzędzi. Inwestycja w badania nad technologiami głębokiego uczenia w medycynie to nie tylko odpowiedź na bieżące potrzeby sektora medycznego, ale także przygotowanie na przyszłość, w której algorytmy uczenia maszynowego stanowią, zgodnie z regulacjami AI Act, integralny element ochrony zdrowia, dzięki wspieraniu lekarzy i zapewniania pacjentom dostępu do szybszej, bardziej precyzyjnej i bardziej spersonalizowanej opieki medycznej.

Rozwój głębokiego uczenia w medycynie nie jest jedynie technologicznym wyzwaniem, ale też moralnym obowiązkiem. W dzisiejszych czasach głównym celem sektora medycznego powinno być odciążenie medyków z pracy, jednocześnie podnosząc jakość opieki zdrowotnej poprzez wcześniejsze wykrywanie chorób i skuteczniejsze leczenie. To jest potencjalny wielki krok dla ludzkości, jeśli podejdziesz się do tych zmian z odpowiednią odpowiedzialnością, zrozumieniem i otwartością.

1.2 Cel i zakres pracy

Celem niniejszej pracy dyplomowej jest analiza wykorzystania algorytmów sztucznej inteligencji do wykrywania zmian w mózgu na podstawie zdjęć uzyskanych z użyciem rezonansu magnetycznego. Praca ta koncentruje się na opracowaniu, ewaluacji i porównaniu modeli uczenia maszynowego zdolnych do zidentyfikowania anomalii występujących w strukturze mózgu, takich jak zmiany patologiczne związane z chorobami neurodegeneracyjnymi i nowotworami mózgu.

W szczególności praca ta będzie skupiała się na analizie obrazów medycznych, w tym przetwarzanie, segmentacja i klasyfikacja obrazów medycznych, wyborem optymalnych modeli i ich walidacji w oparciu o dane kliniczne.

Zakres pracy obejmuje zapoznanie się z teoretycznymi podstawami przetwarzania obrazów, metodami uczenia maszynowego, w szczególności technikami takimi jak transfer learning i metodami klasyfikacji. Wdrożenie poznanych technik z użyciem zaawansowanych narzędzi programistycznych, takich jak skryptowy język programowania Python z bibliotekami PyTorch, NumPy czy Seaborn.

Dodatkowo praca uwzględnia eksploracyjną analizę danych, wstępne przetwarzanie danych oraz wpływ parametrów technicznych na efektywność algorytmów. W ramach oceny modeli wdrożone zostały standardowe miary jakości modelu, takie jak dokładność, precyzja, czułość i F1-Score. Wyniki pracy były analizowane pod kątem potencjalnych ograniczeń, możliwości dalszego rozwoju technologii w kierunku pełnej automatyzacji i integracji z istniejącymi systemami diagnostycznymi.

1.3 Wyzwania związane z tematem pracy

Oparcie pracy o dane medyczne wiąże się z licznymi wyzwaniami, które należy uwzględnić na każdym etapie analizy i przetwarzania danych. Pierwszym, najważniejszym wyzwaniem jest weryfikacja pochodzenia danych. Nie wszystkie zbiory danych dostępne w internecie mają jasno określonych autorów, co powoduje problemy z weryfikacją legalności i rzetelności źródła danych. Natomiast w przypadku danych ze znanymi autorami, lub instytucją zwiększa pewność co do ich autentyczności. Taka transparentność powoduje, że dane najprawdopodobniej zostały odpowiednio zanonimizowane [12] i pacjenci wyrazili zgodę na ich użycie, co jest niezbędne w kontekście ochrony danych osobowych zgodnie z regulacjami takimi jak RODO (GDPR).

Kolejnym wyzwaniem jest zbalansowanie danych. W medycynie wiele schorzeń występuje w różnej częstotliwości, co sprawia, że zbiory danych mogą być silnie niezrównoważone. Przykładowo w zbiorze danych dotyczących nowotworów z klasyfikacją niebinarną, obejmującą różne typy nowotworów, staje się bardziej skomplikowana ze względu na różną ilość danych dla poszczególnego typu. Niezbalansowanie danych może doprowadzić do sytuacji, w której model faworyzuje klasę z największą reprezentacją danych. Sposobem, którym można zaradzić temu problemowi, jest nadpróbkowanie klas, czyli wykorzystanie różnych technik do wyrównania ilości przykładów dla każdej klasy.

Trzecim problemem jest jakość danych, w związku z licznym przetwarzaniem i anonimizacją, dane te są często słabej jakości i pozbawione całego kontekstu medycznego poza diagnozą [12][21]. Brakuje informacji o przebiegu choroby, wcześniejszych wynikach badań, czy danych demograficznych pacjenta, co też nadaje kolejny kontekst do odpowiedniej diagnozy. W efekcie dane, mogą być zbyt uproszczone, aby pozwolić na pełną analizę medyczną lub budowę bardziej

kompleksowych algorytmów uczenia maszynowego.

Dużym problemem jest też interpretowalność wyników przez personel medyczny. Ciężko jest określić dlaczego model jest w stanie poprawnie zakwalifikować odpowiednią klasę, dlatego sieci neuronowe często nazywane są "czarnymi skrzynkami". Przez to osoby, które nie mają na codzień zawodowego doczynienia z tą technologią, mają większe opory co do jej użycia [12][21]. W tym przypadku w celu zapewnienia lepszej interpretowalności wyników, można wprowadzić algorytm, który będzie wskazywał na punkty na obrazie, które spowodowały zakwalifikowanie obrazu do takiej klasy, a nie innej. Poza tym można zacząć edukować medyków na temat tego jak działa sztuczna inteligencja, co można osiągnąć przy jej pomocy. Problem związany ze zrozumieniem dlaczego sieć neuronowa wygenerowała taki wynik jest też regulowany przez AI Act, co znacznie zwiększy zaufanie do korzystania z tego typu narzędzi.

Rozdział 2

Wstęp teoretyczny

2.1 Język programowania

Środowiskiem pracy tej pracy inżynierskiej jest język Python powszechnie używany w dziedzinie Data Science. Do napisania algorytmu głębokiego uczenia użyta została biblioteka PyTorch, do analizy danych biblioteki Matplotlib i Seaborn, a całość została wyuczona w zeszycie Jupyter za pośrednictwem platformy Google Colab.

Python był językiem pierwszego wyboru w pracy ze względu na swoją dużą popularność, nie tylko w środowisku związanym z data science. Według ankiety 2025 Developer Survey, przeprowadzonej przez portal Stackoverflow, język programowania Python jest czwartą najczęściej wybieraną technologią, który wybrało 57,9% ankietowanych [31]. Kiedy mowa o data science, Python jest często wymieniany, jako jedna z kluczowych technologii, które należy opanować, aby rozwijać umiejętności związane z uczeniem maszynowym [1]. Według ankiety portalu 365 Data Science, zrealizowanej na tysiącu ofert pracy, aż 84,6% firm podaje znajomość języka Python jako wymaganą umiejętność na stanowisko data scientist [17].

Python jako technologia nie jest najbardziej zoptymalizowanym rozwiązaniem, więc można zadać pytanie, dlaczego to akurat ten język programowania jest główną technologią, w której pisze się tak obciążające obliczeniowo algorytmy, jak sztuczna inteligencja? Jednym z powodów może być prosta i łatwa do zrozumienia składnia tego języka. W przypadku dziedziny data science, często nie jest wymagane wykształcenie informatyczne, a zamiast tego wykształcenie z dziedzin matematycznych [33]. Prosta składnia języka może powodować, że technologia ta jest szybsza do nauki dla matematyka, niż szybszy język z bardziej skomplikowaną składnią, jakim jest np. C++. Dzięki temu Python oferuje dużo bibliotek do uczenia maszynowego, jak np. NumPy, PyTorch, czy Tensorflow [7].

2.2 Środowisko

Do środowiska uczenia algorytmu został wybrany Google Colab, czyli środowisko obliczeniowe oparte na chmurze, które oferuje dostęp do środowiska z procesorem graficznym NVIDIA TESLA K80 12 GB VRAM [19], bezpłatnie z pewnymi ograniczeniami. Głównym powodem wybrania Google Colab jest zapewnienie natychmiastowego środowiska uruchomieniowego, które nie wymaga instalacji bibliotek (przez co jest to ograniczone do minimum), konfiguracji interpretera ani przygotowywania lokalnej infrastruktury programistycznej.

Google daje dostęp do zasobów obliczeniowych, w tym darmowych jednostek GPU i TPU [8], które są powszechnie wykorzystywane do trenowania modeli głębokiego uczenia. Dzięki temu można trenować modele bez konieczności używania specjalistycznego sprzętu. Jest to szczególnie korzystne do wykorzystania w projektach związanych z głębokim uczeniem, takich jak klasyfikacja obrazów, segmentacja, czy analiza danych medycznych.

Środowisko Google Colab oferuje szeroką integrację z bibliotekami data science, takimi jak numpy, PyTorch, czy Pandas, które są preinstalowane. Dzięki temu można natychmiast przystąpić do implementacji algorytmów, bez konieczności konfigurowania zależności, co powoduje, że jest to przyjazne środowisko do pracy z uczeniem maszynowym [8].

2.3 Biblioteki

2.3.1 Analiza danych

Do analizy danych zostały wykorzystane popularne biblioteki do wizualizacji i obróbki danych. Posłużyły one do wyświetlenia przykładowych obrazów ze zbioru treningowego i testowego, tworzenia wykresów słupkowych przedstawiających liczebność poszczególnych klas, a także do wygenerowania map cieplnych służących ocenie oraz porównaniu wydajności testowanych modeli.

2.3.2 Modele uczenia maszynowego

Do wykonania modeli została wykorzystana biblioteka PyTorch i Torchvision ze względu na ich właściwości, które znacząco ułatwiają pracę z modelami głębokiego uczenia oraz przetwarzaniem obrazów. PyTorch to biblioteka opierająca się na dynamicznym grafie obliczeniowym (define-by-run) [22], który tworzony jest w czasie wykonywania programu. Dzięki temu możliwe jest elastyczne modelowanie architektury modeli, łatwe debugowanie oraz dostosowywanie kodu do niestandardowych zadań [16].

PyTorch zapewnia wydajną i intuicyjną obsługę procesora graficznego, co jest ważnym elementem pracy przy głębokich sieciach neuronowych. Integracja z CUDA znacznie przyspiesza obliczenia o skraca czas treningu modeli, zachowując przy tym składnię przenoszenia tensora między procesorem obliczeniowym, a procesorem graficznym [35].

Torchvision jest istotną częścią projektu dotyczącą przetwarzania obrazów. Biblioteka ta rozszerza możliwości PyTorch o zestaw narzędzi dedykowanych zadaniom komputerowego rozpoznawania obrazu. Zawiera ona gotowe metody przetwarzania wstępnego obrazów i augmentacji danych, takie jak normalizacja, zmiana rozmiaru, czy losowe przekształcenia, upraszczając przygotowanie danych do treningu [25]. Dodatkowo dostarcza gotowe struktury zbiorów danych [23].

Biblioteka torchvision umożliwia dostęp do pretrenowanych modeli, takich jak ResNet, czy VGG, które dostarczane są wraz z wstępnie wytrenowanymi wagami, co umożliwia efektywne zastosowanie transfer learningu [26]. Transfer learning stosowany jest w celu skrócenia czasu i poprawienia wyników algorytmu. Dzięki temu możliwe jest sprawne prototypowanie architektur oraz wdrażanie sprawdzonych rozwiązań, co przyspiesza rozwój projektów [13].

2.3.3 Graficzny interfejs użytkownika

Do zaprezentowania działania algorytmu wykorzystane zostały biblioteki Gradio, OpenCV i Pillow. Każda z nich spełnia odmienną rolę i uzupełnia pozostałe, umożliwiając stworzenie intuicyjnego i interaktywnego środowiska demonstracyjnego.

Biblioteka Gradio posłużyła do przygotowania prostego w obsłudze interfejsu użytkownika dostępnego z poziomu przeglądarki internetowej. Dzięki tej bibliotece możliwe było szybkie utworzenie interaktywnego demo, w którym użytkownik może wgrywać obrazy, obserwować wyniki przetwarzania oraz testować działanie algorytmu bez konieczności instalacji dodatkowego oprogramowania. Gradio automatyzuje również proces uruchamiania lokalnego serwera, oraz generowania interaktywnych komponentów, takich jak pola wejściowe, podglądy obrazów, czy pola tekstowe.

Biblioteka OpenCV (Open Source Computer Vision Library) została głównie wykorzystana do przetwarzania obrazów. Biblioteka ta oferuje szeroki zakres funkcji umożliwiających m.in. odczyt, modyfikację, filtrację, binaryzację i analizę obrazów.

Biblioteka Pillow pełni funkcję pomocniczą przy operacjach na obrazach, szczególnie w zakresie konwersji między różnymi formatami i typami danych. Główne jej zastosowanie w programie jest integracja obrazów z interfejsem graficznym Gradio.

2.4 Odwołania do literatury

W medycynie jest coraz większe zapotrzebowanie na wsparcie pracy personelu medycznego. W styczniu 2024 roku portal Fierce Biotech opublikował artykuł dotyczący dopuszczenia przez amerykańską rządową agencję FDA (Food and Drug Administration) algorytmu sztucznej inteligencji służącej do diagnostyki choroby Alzheimera za pomocą obrazu z rezonansu magnetycznego. Algorytm nazywa się BrainSee i został opracowany przez firmę Darmiyan, Inc [9].

2.4.1 Artykuły naukowe

W związku z popularnością tworzenia zaawansowanych systemów wizji komputerowej do zastosowań medycznych ilość artykułów na temat detekcji zmian na podstawie obrazów rezonansu magnetycznego.

Autorzy artykułu dotyczącego wykrywania zmian związanych ze stwardnieniem rozsianym na podstawie zdjęć rezonansu magnetycznego przedstawiają porównanie kilku rozwiązań: w pełni połączonych sieci neuronowych (zarówno dwuwymiarowych, jak i trójwymiarowych), spłotowych sieci neuronowych, pretrenowanych sieci spłotowych (np. ResNet), sieci U-Net, sieci GAN, autoenkoderów i hybrydowej architektury łączącej sieci spłotowe z rekurencyjnymi sieciami neuronowymi. Najlepsze osiągnięcia ma architektura dwuwymiarowej spłotowej sieci neuronowej z dokładnością na poziomie 99,05% [29].

W literaturze można znaleźć też artykuły dotyczące algorytmów uczenia bez nadzoru. Takie podejście autorzy zaproponowali w celu automatycznego wykrywania stwardnienia rozsianego w mózgu na podstawie zdjęć rezonansu magnetycznego z kontrastem. Autorzy wyszukują zmian za pomocą nieliniowej redukcji wymiarów, w tym przypadku IsoMap i lokalne liniowe osadzanie. W wyniku działania algorytmów, zamiast trójwymiarowego zdjęcia otrzymano jedno zdjęcie spłaszczone do dwóch wymiarów, co spowodowało podkreślenie zmian. Algorytmy były ewaluowane za pomocą miary podobieństwa Dice'a. Lokalne liniowe osadzanie poradziło sobie na poziomie $74 \% \pm 1 \%$, a algorytm IsoMap na poziomie $78 \% \pm 9 \%$ [34].

W celu przetwarzania obrazów rezonansu magnetycznego jednymi z najczęściej używanych algorytmów, do analizy pod kątem choroby Alzheimera, jest maszyna wektorów wspierających, niesplotowe sztuczne sieci neuronowe, spłotowe sieci neuronowe, regresja logistyczna. Najbardziej efektywne są w tym celu spłotowe sieci neuronowe, ponieważ z wyżej przedstawionych modeli mają najwyższą dokładność zaczynając od 72%, a na 99% dokładności kończąc [5].

W przypadku analizy obrazów rezonansu magnetycznego w kontekście guzów mózgu, gdzie wykorzystano spłotowe sieci neuronowe i algorytm YOLOv7 do detekcji, autorzy wskazują na

dokładność na poziomie 99,5%, w przypadku klasyfikacji binarnej. W porównaniu do popularnego ResNet50, które ma dokładność na poziomie 96,5% i VGG16, które ma dokładność na poziomie 97,6% [14].

W literaturze naukowej można znaleźć też informacje na temat użycia sieci YOLOv7 do wykrywania zmian z czterema klasami. Autorom udało się uzyskać dokładność na poziomie 99,5%, a precyzji i czułości na poziomie 99,3%. Drugim najlepszym modelem wskazanym przez autorów jest EfficientNet z precyzją na poziomie 97,7% i czułością na poziomie 97,9%. Minimalnie gorszy okazał się model VGG16 z precyzją na poziomie 97,4% i czułością na poziomie 97,7% [2].

Nie brakuje też literatury dotyczącej sieci YOLOv8. Autorzy wytrenowali ulepszony algorytm YOLOv8 używając dwustu epok, stałej uczącej na poziomie jednej setnej i regularyzacji wag na poziomie pięciu tysięcznych, a dane były niebinarnymi klasami. Całość była trenowana na karcie graficznej NVidia Tesla T4 15GB VRAM. Autorzy wskazują, że wartość *mean Average Precision* w przypadku ich algorytmu wynosiła 91%, w porównaniu do zwykłego YOLOv8 na poziomie 87% [3].

W artykule dotyczącym wykrywania guzów w mózgu za pomocą spłotowych sieci neuronowych, autorzy porównują różne architektury tych sieci i porównują ich jakość na podstawie pięciu miar: dokładność, czułość, swoistość, precyzja i F1-Score. Według autorów modele uczą się najlepiej z niską stałą uczącą i z umiarkowaną wielkością wsadu. Zoptymalizowane przez autorów sieci spłotowe, co prawda wykazują się niższą dokładnością, niż inne zaproponowane architektury, ale za to charakteryzują się wyższą czułością na poziomie 99,2% [18].

Modeli pretrenowanych sieci spłotowych można też używać w zadaniach związanych z segmentacją zmian nowotworowych. Autorzy artykułu porównują jakość spłotowych sieci neuronowych, VGG16, AlexNet i GoogleNet. Ze wszystkich sieci najlepiej poradziła sobie sieć VGG16 z dokładnością na poziomie 85,68% przy uczeniu na sześćdziesięciu epokach. Autorzy zwracają uwagę na to, że model może poradzić sobie lepiej przy użyciu bardziej zróżnicowanych zdjęć MRI oraz przy użyciu innych rozwiązań opartych na spłotowych sieciach neuronowych [6].

Inną siecią, która posiada imponujące wyniki, jest ResNet. Autor proponuje dostosowany model ResNet34. Wykorzystane dane miały cztery niezbalansowane klasy. Model ten był trenowany na pięćdziesięciu epokach z entropią skrośną jako funkcję straty, funkcją optymalizacyjną Ranger, przyjętą stałą uczącą na poziomie jednej tysięcznej, a rozmiar wsadu na poziomie trzydziestu dwóch próbek. Całość uczenia trwała 37 minut. Miarami jakości modelu były dokładność, precyzja, czułość i F1-Score. Dokładność, precyzja i F1-Score były na poziomie 99,66%, a czułość na poziomie 99,67%. Autor porównuje działanie modelu z innymi modelami i drugim najlepszym modelem był NASNet z dokładnością na poziomie 99,6% [27].

Natomiast autorzy zbioru danych wykorzystanych w pracy przedstawia za najskuteczniejszy transfer learning z użyciem sieci VGG16 z dokładnością na poziomie 94% na zbiorze walidacyjnym i F1-Score na poziomie 91% [15].

Rozdział 3

Prezentacja i omówienie wyników pracy

3.1 Klasyfikacja

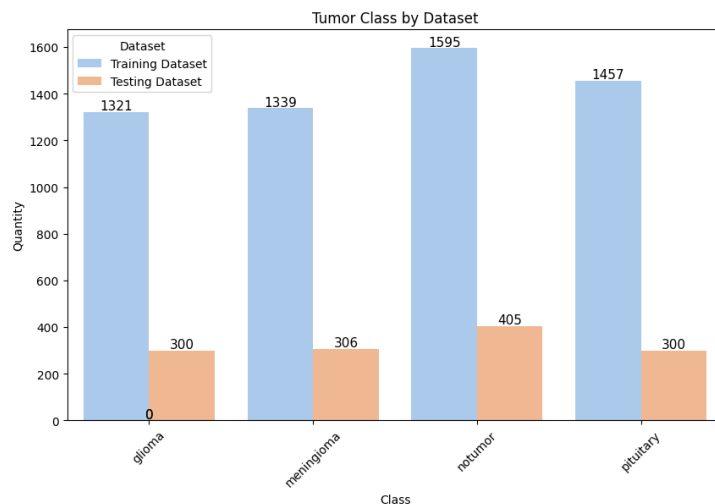
3.1.1 Eksploracyjna analiza danych

Dane

Do treningu modeli został użyty czteroklasowy, niezbalansowany zbiór danych [15]. Dane udostępnione są na licencji MIT. Zbiór jest podzielony na podzbiór treningowy i testowy. Zbiór treningowy zawiera łącznie 5712 zdjęć mózgu, w tym 1321 zdjęć z obecnym glejakiem, 1339 zdjęć z obecnym oponiakiem, 1457 zdjęć z guzem przysadki mózgowej i 1595 zdjęć mózgu bez widocznych guzów. Zbiór testowy zawiera 1311 zdjęć mózgu, w tym 300 zdjęć z obecnym glejakiem, 306 zdjęć z obecnym oponiakiem, 300 zdjęć z guzem przysadki mózgowej i 405 zdjęć mózgu bez widocznych guzów.

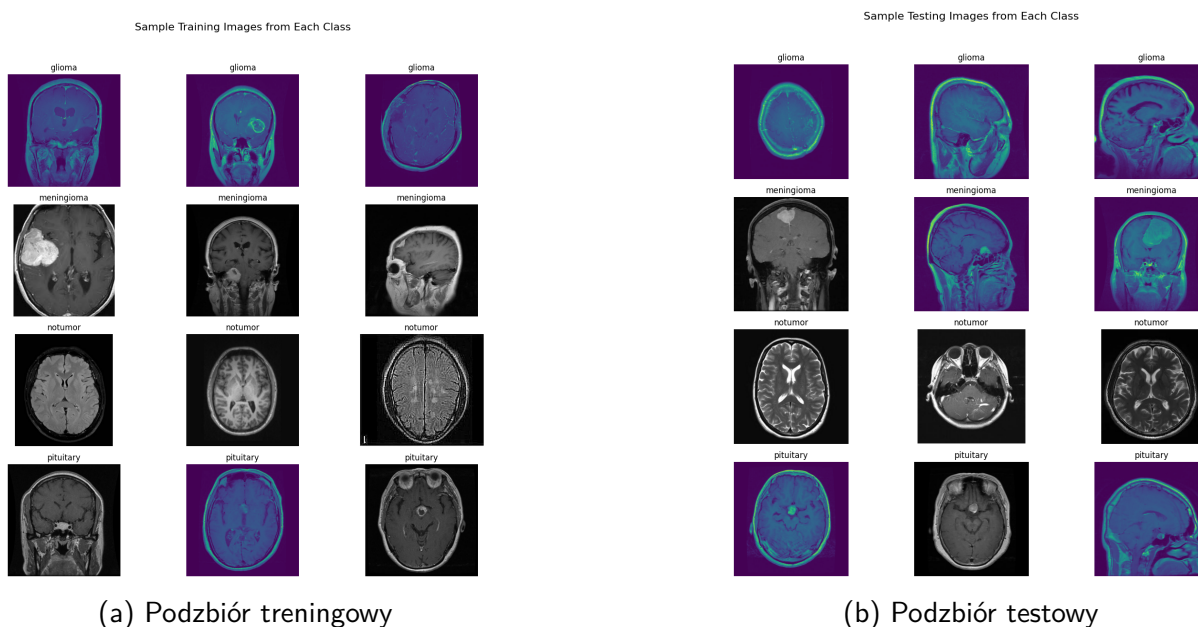
Analiza danych

Na podstawie wyżej wymienionych informacji, za pomocą biblioteki seaborn i modułu pyplot biblioteki matplotlib, został utworzony wykres słupkowy, z podziałem na klasy reprezentujący podzbiór treningowy i testowy (Rys. 3.10).



Rysunek 3.1: Wykres słupkowy z rozkładem danych

Po utworzeniu wykresu słupkowego, za pomocą modułu image biblioteki matplotlib i biblioteki random zostały utworzone po trzy próbki zdjęć z każdej klasy. Biblioteka random posłużyła do wybrania trzech losowych zdjęć każdej klasy z podzbioru treningowego i testowego.



Rysunek 3.2: Przykładowe zdjęcia ze zbioru danych

Dzięki utworzeniu próbek można zauważyć (Rys. 3.2), że dane poza tym, że są niezbalansowane ilościowo, są też przedstawione z perspektywy różnych osi w mózgu, co ma swoje zalety, ale też i wady. Dużą zaletą tak zróżnicowanej reprezentacji danych jest to, że model ma możliwość nauczenia się wyglądu mózgu z różnych perspektyw, co przekłada się na lepsze możliwości jego praktycznego zastosowania. Natomiast wadą jest to, że przez tak zróżnic-

wane dane model może gorzej się uczyć, co powoduje, że model może osiągać gorsze wyniki w uczeniu. Biorąc pod uwagę fakt, że bardziej istotne jest to, aby móc model zastosować w różnych przypadkach w praktyce, to zalety są większym argumentem, niż wady. Problem związany z tak dużym zróżnicowaniem danych może być skorygowany dobierając inne hiperparametry, poprawiając jakość modelu.

3.1.2 Trening modeli

Transformacje

Przed przejściem do treningu modeli należy dokonać transformacji. Trening był wykonywany techniką transfer learningu, czyli metoda uczenia sieci maszynowych, która wykorzystuje pretrenowane sieci neuronowe do wyspecyfikowanego zadania. Pretrenowane sieci neuronowe mają zdefiniowaną wielkość wejścia, którą jest 224 na 224 pikseli w skali RGB, więc do tej wielkości trzeba dostosować zdjęcia.

Do dokonania transformacji wykorzystany został współczesny moduł v2 z biblioteki torchvision. Został on wybrany zamiast klasycznego modułu transforms ze względu na lepsze wsparcie dla tensorów [24].

Poza dostosowaniem rozmiaru do wejścia przetrenowanych sieci neuronowych, wykorzystane zostały też funkcje związane z losowymi zmianami pozycji zdjęcia, takie jak przereźnięcie horyzontalne, przereźnięcie wertykalne i obrócenie zdjęcia o 10 stopni. Po przetłumaczeniu obrazu na tensor została dokonana normalizacja typowa dla pretrenowanych sieci neuronowej.

Dla zbioru testowego zostały dokonane jedyne zmiany związane z rozmiarem zdjęć, przetłumaczenie obrazu na tensor i normalizacja. Transformacje zostały wprowadzone na zdjęcia za pomocą metody ImageFolder z modułu datasets biblioteki torchvision.

Trening

Hiperparametry

Po transformacjach zostały zdefiniowane dwa hiperparametry. Rozmiar wsadu został ustalony na 64 próbki, a stała ucząca została ustalona na 0,0001. Rozmiar wsadu wykorzystany był w metodzie DataLoader, która automatyzuje i usprawnia wczytywanie danych podczas trenowania modeli. Funkcją straty wykorzystaną w treningu była entropia skrośna, a jako algorytm optymalizacji został wykorzystany Adam. Obie funkcje są już zdefiniowane w bibliotece PyTorch. Do metody Adam przy jej definiowaniu została podana stała ucząca. Wszystkie modele uczyły się na 20 epokach.

Miary jakości modelu

W celu mierzenia jakości modelu podczas treningu i testowania, zostały wykorzystane cztery miary: dokładność, precyzja, czułość i F1-Score. Dokładność to miara, która określa to ile razy model się pomylił. Biorąc to pod uwagę, jest to dość słaba miara dla danych niezbalansowanych. Precyzja określa jaka część z przewidzianych wartości pozytywnych, to wartości prawdziwie pozytywne. Czułość określa jaka część z wszystkich prawdziwych wartości, to wartości prawdziwie pozytywne. F1-Score to średnia harmoniczna precyzji i czułości.

$$\text{Dokładność} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Czułość} = \frac{TP}{TP + FN}$$

$$\text{Precyzja} = \frac{TP}{TP + FP}$$

$$F1 - \text{Score} = \frac{2}{\frac{1}{\text{Precyzja}} + \frac{1}{\text{Czułość}}} = 2 \cdot \frac{\text{Precyzja} \cdot \text{Czułość}}{\text{Precyzja} + \text{Czułość}}$$

We wzorach TP (*True Positives*) oznacza liczbę przypadków prawdziwie pozytywnych, TN (*True Negatives*) oznacza liczbę przypadków prawdziwie negatywnych, FP (*False Positives*) oznacza liczbę przypadków fałszywie pozytywnych, a FN (*False Negatives*) oznacza liczbę przypadków fałszywie negatywnych.

W celu obliczenia dokładności zostały podzielone wartości prawidłowe podzielone przez wszystkie wartości. Do obliczenia czułości i precyzji zostały wykorzystane odpowiednie metody z modułu scikit-learn. W celu obliczenia wartości F1-score została napisana funkcja przyjmująca w argumentach precyzję i czułość, a zwracająca ich średnią harmoniczną. Na teście metryki były obliczone za pomocą metody z biblioteki scikit-learn zwracającej wyżej wymienione metryki dla poszczególnych klas i wyciągającą z nich średnią. Po teście za pomocą bibliotek matplotlib i seaborn, została wygenerowana mapa cieplna, która reprezentuje to do jakiej klasy model przepisał dane zdjęcie.

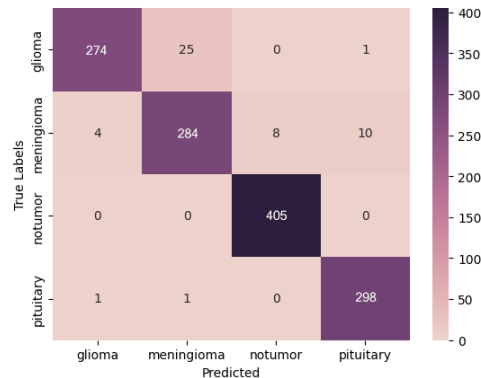
Sieci VGG

Sieć VGG to proste, głębokie sieci splotowe zbudowane z wielu powtarzalnych bloków Conv 3x3 + ReLU + MaxPool, które osiągnęły dobre wyniki na ImageNet Challenge 2014 dzięki konsekwentnie stosowanej architekturze. Ich dobre wyniki polegają na głębokości oraz użyciu małych filtrów co pozwala efektywnie wydobywać coraz bardziej złożone cechy z obrazu [30].

VGG16

W tym eksperymencie sieć VGG16 osiągnęła na treningu 97,79% dokładności, czułość na poziomie 83,33%, precyzję na poziomie 90,83%, F1-score na poziomie 86,92% z wynikiem funkcji straty na poziomie 0,063693.

Sieć VGG16 na teście osiągnęła dokładność na poziomie 96%, czułość na poziomie 96%, precyzję na poziomie 96% i F1-score na poziomie 96%.

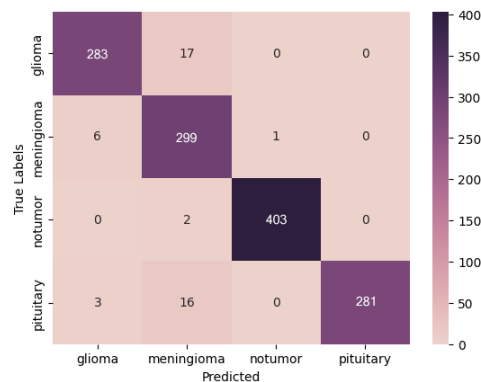


Rysunek 3.3: Mapa cieplna sieci VGG16

VGG19

W tym eksperymencie sieć VGG19 osiągnęła na treningu 97,86% dokładności, czułość na poziomie 100%, precyzję na poziomie 100%, F1-score na poziomie 100% z wynikiem funkcji straty na poziomie 0,057244.

Sieć VGG19 na teście osiągnęła dokładność na poziomie 97%, czułość na poziomie 97%, precyzję na poziomie 97% i F1-score na poziomie 97%.



Rysunek 3.4: Mapa cieplna sieci VGG16

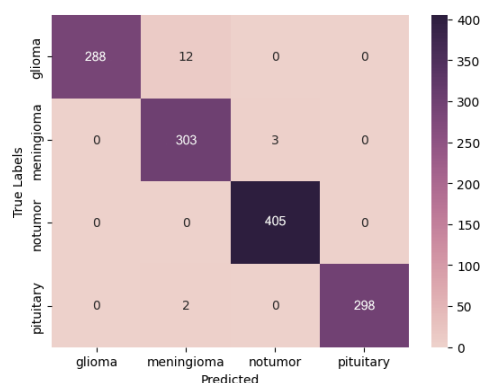
Sieci ResNet

Sieci ResNet to głębokie sieci splotowe wykorzystujące połączenia rezydualne, które omijają część warstw, dzięki czemu model się łatwiej uczy i unika problemu zanikającego gradientu. Dzięki temu mogą być to bardzo głębokie sieci i osiągać wysoką dokładność w zadaniach wizji komputerowej [11].

ResNet18

W tym eksperymencie sieć ResNet18 osiągnęła na treningu 99,86% dokładności, czułość na poziomie 100%, precyzję na poziomie 100% i F1-score na poziomie 100% z wynikiem funkcji straty na poziomie 0,005534.

Sieć ResNet18 na teście osiągnęła dokładność na poziomie 99%, czułość na poziomie 99%, precyzję na poziomie 99% i F1-score na poziomie 99%.

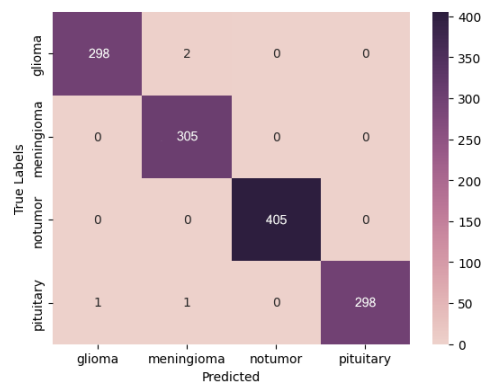


Rysunek 3.5: Mapa cieplna sieci ResNet18

ResNet50

W tym eksperymencie sieć ResNet50 osiągnęła na treningu 99,84% dokładności, czułość na poziomie 100%, precyzję na poziomie 100% i F1-score na poziomie 100% z wynikiem funkcji straty na poziomie 0,005111.

Sieć ResNet50 na teście osiągnęła dokładność na poziomie 100%, czułość na poziomie 100%, precyzję na poziomie 100% i F1-score na poziomie 100%.

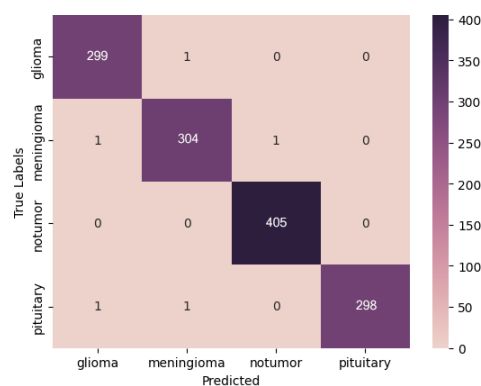


Rysunek 3.6: Mapa cieplna sieci ResNet50

ResNet101

W tym eksperymencie sieć ResNet101 osiągnęła na treningu 99,74% dokładności, czułość na poziomie 95%, precyzję na poziomie 93,75% i F1-score na poziomie 94,37% z wynikiem funkcji straty na poziomie 0,011064.

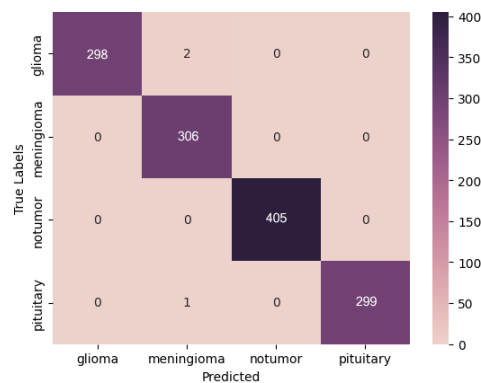
Sieć ResNet101 na teście osiągnęła dokładność na poziomie 100%, czułość na poziomie 100%, precyzję na poziomie 100% i F1-score na poziomie 100%.



Rysunek 3.7: Mapa cieplna sieci ResNet101

ResNet152

W tym eksperymencie sieć ResNet152 osiągnęła na treningu 99,91% dokładności, czułość na poziomie 100%, precyzję na poziomie 100% i F1-score na poziomie 100% z wynikiem funkcji straty na poziomie 0,004163. Sieć ResNet152 na teście osiągnęła dokładność na poziomie 100%, czułość na poziomie 100%, precyzję na poziomie 100% i F1-score na poziomie 100%.



Rysunek 3.8: Mapa cieplna sieci ResNet152

3.1.3 Porównanie

Model	Funkcja Straty	Dokładność	Czułość	Precyzja	F1-Score
VGG16	0,063693	97,79%	83,33%	90,83%	86,92%
VGG19	0,057244	97,86%	100%	100%	100%
ResNet18	0,005534	99,86%	100%	100%	100%
ResNet50	0,005111	99,84%	100%	100%	100%
ResNet101	0,011064	99,74%	95%	93,75%	94,37%
ResNet152	0,004163	99,91%	100%	100%	100%

Tabela 3.1: Tabela porównawcza wyników treningu poszczególnych modeli

Model	Dokładność	Czułość	Precyzja	F1-Score
VGG16	96%	96%	96%	96%
VGG19	97%	97%	97%	97%
ResNet18	99%	99%	99%	99%
ResNet50	100%	100%	100%	100%
ResNet101	100%	100%	100%	100%
ResNet152	100%	100%	100%	100%

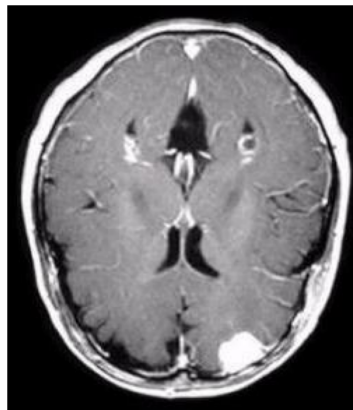
Tabela 3.2: Tabela porównawcza wyników testu poszczególnych modeli

Jak widać w wartościach podanych w tabeli, sieci ResNet zdecydowanie lepiej poradziły sobie z zadaniem klasyfikacji guzów mózgu. Biorąc pod uwagę to, że model nie był nieomylny, wartości zwracane przez metody obliczające miary jakości modelu zostały jedynie przybliżone do wyniku zbliżonego do 100%, ponieważ pomyłki można liczyć w promilach, co widać na rysunkach 3.6, 3.7 i 3.8.

3.2 Detekcja

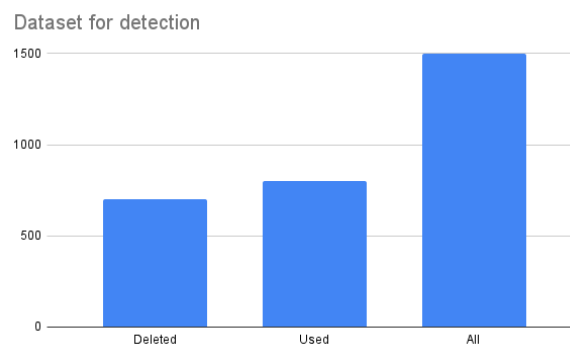
3.2.1 Eksploracyjna analiza danych

Do utworzenia modelu detekcji guzów mózgu został wykorzystany publiczny zbiór danych Br35H :: Brain Tumor Detection 2020 [10]. Zbiór danych zawiera 1500 różnych zdjęć rezonansu magnetycznego guzów mózgu i plik .json w którym znajdują się informacje na temat tego, gdzie guz się znajduje.



Rysunek 3.9: Przykładowe zdjęcie ze zbioru danych Br35H: klasa z obecnym guzem mózgu

Biorąc pod uwagę to, że nie wszystkie zdjęcia zawierają informacje na temat położenia guza, to trzeba było wyczyścić dane ze zdjęć, które nie zawierają informacji na temat położenia guza. W wyniku czyszczenia danych zostało usuniętych 699 zdjęć, pozostawiając do uczenia 801 zdjęć.



Rysunek 3.10: Wykres słupkowy pokazujący ilość użytych i usuniętych danych

Dostosowanie danych do zadania detekcji polegało na utworzeniu prostokątnych ramek. W tym celu należało przetłumaczyć dokładne współrzędne guzów na ramki o odpowiednim

kształcie, a następnie przeliczenie ich na format YOLO. Przy tworzeniu współrzędnych ramek należało wziąć pod uwagę, że guzy występują w zbiorze danych w trzech różnych kształtach: okrągłym, eliptycznym i w formie wielokąta. Po utworzeniu etykiet, które zawierają współrzędne guzów utworzony został przykład losowych zdjęć ze zbioru danych.

3.2.2 Trening modelu YOLOv8

Model YOLOv8 został opublikowany przez Ultralytics 10 stycznia 2023 roku. Model You Only Look Once v8 wykorzystuje lekką i szybką architekturę, osiągając wysoką dokładność przy małej ilości czasu działania algorytmu powodując, że jest dobrym wyborem do praktycznych zastosowań w czasie rzeczywistym [32]. Do uczenia modelu było wykorzystanych 500 próbek, a do testu 301 próbek. Model był trenowany pod możliwość detekcji jednej klasy, czyli guzów mózgu.

Miarą jakości modelu była metryki mean average precision, czyli mAP50 i mAP50-95. Metryka mAP50 to miara jakości detekcji obiektów, która ocenia, czy przewidziany obiekt pokrywa się z prawdziwym co najmniej w 50%, a mAP50-95 to miara jakości detekcji obiektów sprawdzającą poprawność modelu na wielu poziomach, od tak łatwych jak mAP50, ale też bardziej wymagających, zwracając średnią z wszystkich miar. Funkcją optymalizacyjną w treningu modelu był AdamW, ze stałą uczącą 0,002. Są to wartości automatycznie zaproponowane przez bibliotekę ultralytics.

Uczenie modelu zakończyło się z mAP@50 na poziomie 92,3% i mAP@50-95 na poziomie 65,6% podczas treningu. Model podczas testowania osiągnął miarę mAP@50 równą 92,3%, a mAP@50-95 równą 65,7%. Po wytrenowaniu modelu model został przetestowany na przykładowym zdjęciu rezonansu magnetycznego mózgu ze zbioru danych użytego w klasyfikacji. Model wykrył guza z pewnością 81%.

3.3 Graficzny Interfejs Użytkownika

W celu zaprezentowania działania modeli został utworzony graficzny interfejs użytkownika za pomocą biblioteki Gradio. Interfejs użytkownika dzieli się na trzy części, które podzielone są na zakładki: GradioCam, OpenCV i YOLOv8. Zakładki znajdują się w lewym górnym rogu strony.

Pierwsza i druga zakładka mają identyczny interfejs użytkownika. Na górze strony jest miejsce do dodania zdjęcia, poniżej tego pola znajdują się dwie kolumny, każda ma po 6 rzędów. Lewa kolumna służy do wyświetlenia zdjęcia, a prawa do wyświetlenia etykiet klas z podanym

prawdopodobieństwem przyznanych przez każdą sieć. Pod kolumnami znajduje się przycisk, który powoduje uruchomienie programu.

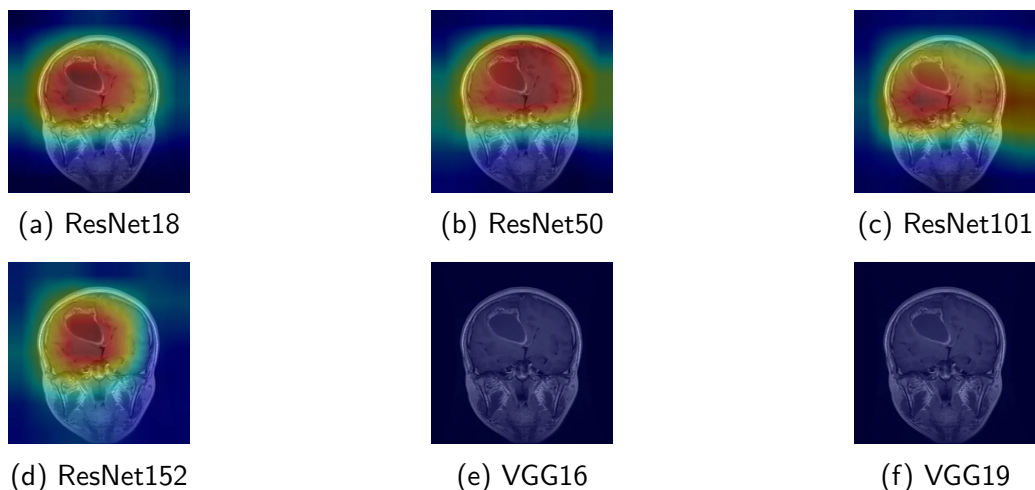
Trzecia zakładka na górze strony posiada pole do dodania zdjęcia, a poniżej tego znajduje się jedna kolumna z ośmioma rzędami. W pierwszym rzędzie znajduje się pole służące do wyświetlenia zdjęcia z wynikiem działania algorytmu YOLOv8. Drugi rząd wyświetla etykietę z informacją o dokładności działania algorytmu YOLOv8. Pozostałe sześć rzędów wyświetla etykiety w podobny sposób jak prawa kolumna w pierwszych dwóch zakładkach.

3.4 Działanie modeli w praktyce

3.4.1 Grad-CAM

Grad-CAM (Gradient-weighted Class Activation Mapping) to technika, która służy do wizualizacji, które fragmenty obrazu mają największy wpływ na decyzję podejmowaną przez model. W celu zwizualizowania działania modelu została wybrana ostatnia warstwa spłotowa sieci, obliczone gradienty i utworzona mapa aktywacji. W przypadku klasy 2, która oznacza zdjęcie bez widocznego guza zwracana jest pusta mapa aktywacji.

Klasa 0: Glejak

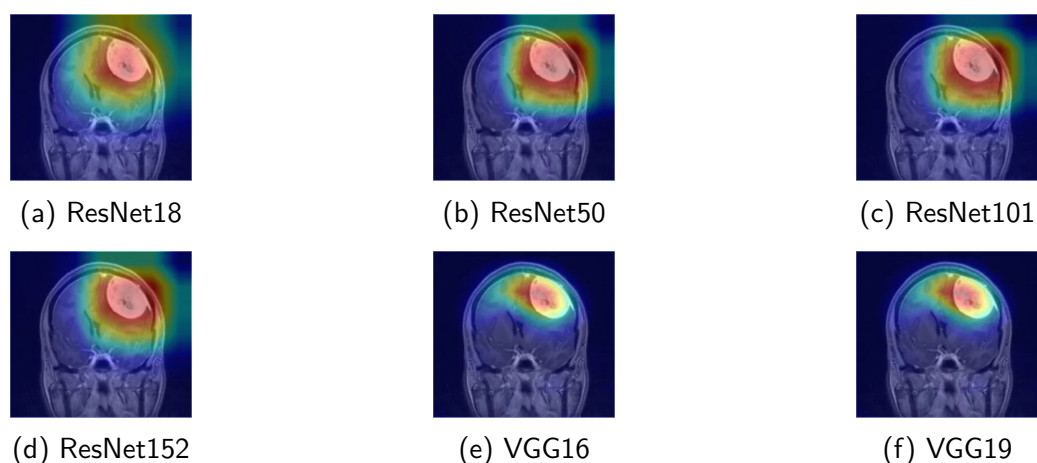


Rysunek 3.11: Wyniki działania sieci z Grad-CAM'em dla klasy 0: glejak

W przypadku klasy 0, czyli zdjęć rezonansu magnetycznego mózgu z widocznym glejakiem sieci ResNet pokazały miejsce znajdowania się guza bardzo ogólnie, a dla sieci VGG nie udało się wygenerować mapy aktywacji.

Poza zwróceniem obrazu z mapą aktywacji, modele w programie zwracają etykietę z nazwą klasy i prawdopodobieństwem, z jakim ta klasa występuje. W przypadku zaprezentowanych zdjęć (Rys. 3.11), prawie wszystkie sieci pokazały prawdopodobieństwo klasy równe 100%, poza modelem VGG16, które pokazało prawdopodobieństwo równe 99,99%, co jest niewielką różnicą.

Klasa 1: Oponiak

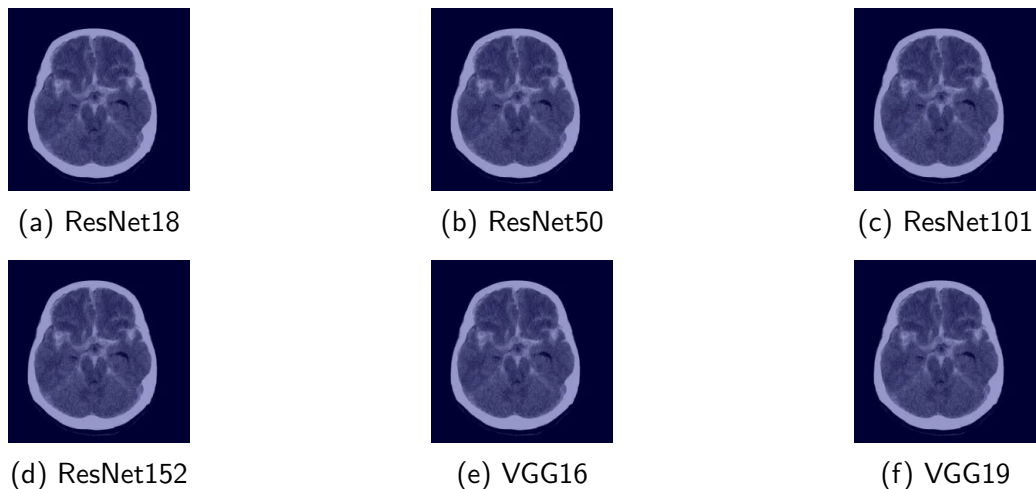


Rysunek 3.12: Wyniki działania sieci z Grad-CAM'em dla klasy 1: oponiak

W przypadku klasy 1, czyli zdjęć rezonansu magnetycznego mózgu z widocznym oponiakiem, na podstawie wszystkich zaproponowanych sieci udało się wygenerować dokładną mapę aktywacji. Sieci VGG okazały się w tym przypadku bardziej dokładne, niż sieci ResNet (Rys. 3.12).

Zwrócone etykiety pokazują, że to jednak wyniki sieci ResNet są bardziej dokładne, ponieważ wszystkie sieci ResNet zwróciły prawdopodobieństwo wystąpienia klasy oponiak równe 100%. Sieć VGG16 zwróciła prawdopodobieństwo równe 96,19%, a sieć VGG19 zwróciła prawdopodobieństwo 88,49%.

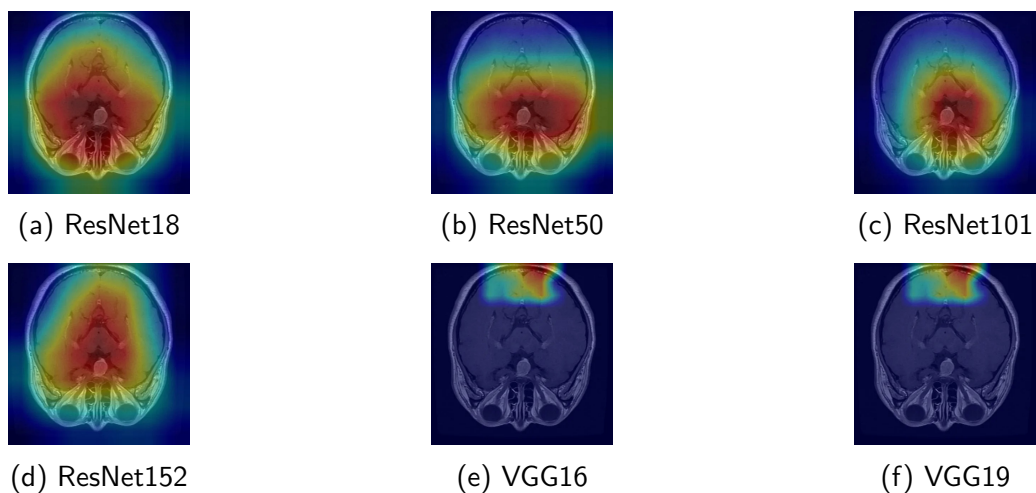
Klasa 2: Brak guza



Rysunek 3.13: Wyniki działania sieci z Grad-CAM'em dla klasy 2: Brak guza

W przypadku braku guza program zwraca pustą mapę aktywacji. W tym przypadku, podobnie jak w przypadku klasy 0, większość modeli zwróciło 100% prawdopodobieństwo braku widocznego guza. Wyjątkiem jest model ResNet18, który zwrócił niewiele niższe prawdopodobieństwo wynoszące 99,98%.

Klasa 3: Guz przysadki mózgowej



Rysunek 3.14: Wyniki działania sieci z Grad-CAM'em dla klasy 3: Guz przysadki mózgowej

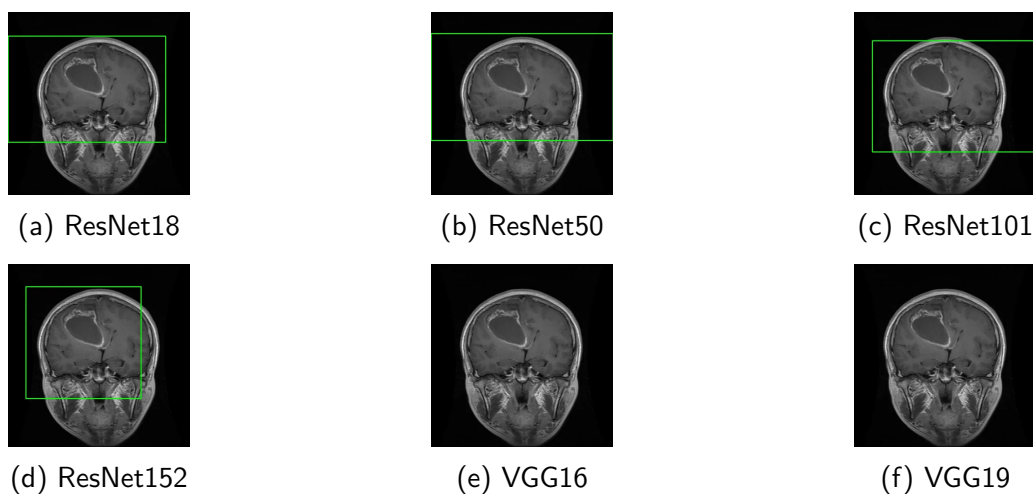
Podobnie jak w przypadku klasy 0, na podstawie sieci udało się wygenerować mało dokładną mapę aktywacji w miejscu, w którym widoczny jest guz. Mapy dobrze pokazują miejsce,

w którym guz się znajduje, ale pokazują też dużo miejsc zdrowych. Natomiast sieci VGG nie zwróciły poprawnej mapy aktywacji, co jest widoczne na rysunkach (Rys. 3.14).

W przypadku zwróconych etykiet, to modele ResNet18, ResNet152 i VGG19 zwróciły prawdopodobieństwo 99,99% występowania klasy 3, a pozostałe modele zwróciły prawdopodobieństwo równe 100%.

3.4.2 Ramki OpenCV

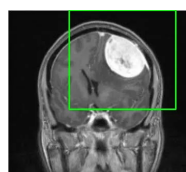
Klasa 0: Glejak



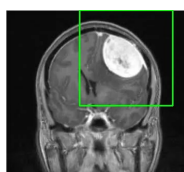
Rysunek 3.15: Wyniki działania sieci z ramkami wygenerowanymi przez bibliotekę OpenCV dla klasy 0: Glejak

Podobnie jak w przypadku Grad-CAMa, na podstawie sieci VGG nie dało rady wygenerować ramek, a ramki wygenerowane na podstawie są mało dokładne. Funkcja dobrze wskazała miejsce występowania guza, ale przez ilość wskazanego zdjęcia, nie jest to dokładna reprezentacja. Dzięki generowaniu ramek zamiast mapy aktywacji można lepiej zauważyć, że sieć bierze wygląd całego mózgu pod uwagę w przypadku tej klasy, a nie samą obecność guza. W związku z brakiem zmian w strukturze przewidywania etykiet, one pozostają bez zmian.

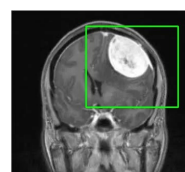
Klasa 1: Oponiak



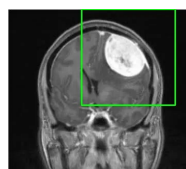
(a) ResNet18



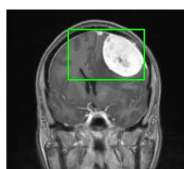
(b) ResNet50



(c) ResNet101



(d) ResNet152



(e) VGG16



(f) VGG19

Rysunek 3.16: Wyniki działania sieci z ramkami wygenerowanymi przez bibliotekę OpenCV dla klasy 1: Oponiak

Dla klasy 1, czyli zdjęć rezonansu magnetycznego z widocznym oponiakiem, ramki wygenerowały się bez problemu dla sieci ResNet i VGG. W przypadku tej klasy, tak samo jak było w przypadku mapy aktywacji, na podstawie sieci VGG udało wygenerować się dokładniejsze ramki

Klasa 2: Brak guza



(a) ResNet18



(b) ResNet50



(c) ResNet101



(d) ResNet152



(e) VGG16

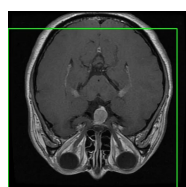


(f) VGG19

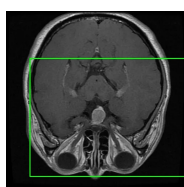
Rysunek 3.17: Wyniki działania sieci z ramkami wygenerowanymi przez bibliotekę OpenCV dla klasy 2: Brak guza

W przypadku, kiedy model sklasyfikuje wprowadzone zdjęcie, jako klasę 2, czyli brak widocznego guza na zdjęciu rezonansu magnetycznego, to ramki nie są generowane i zwracane jest wprowadzone zdjęcie.

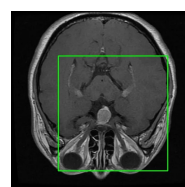
Klasa 3: Guz przysadki mózgowej



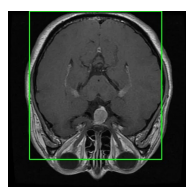
(a) ResNet18



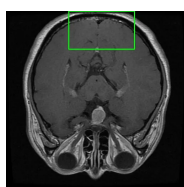
(b) ResNet50



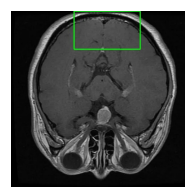
(c) ResNet101



(d) ResNet152



(e) VGG16

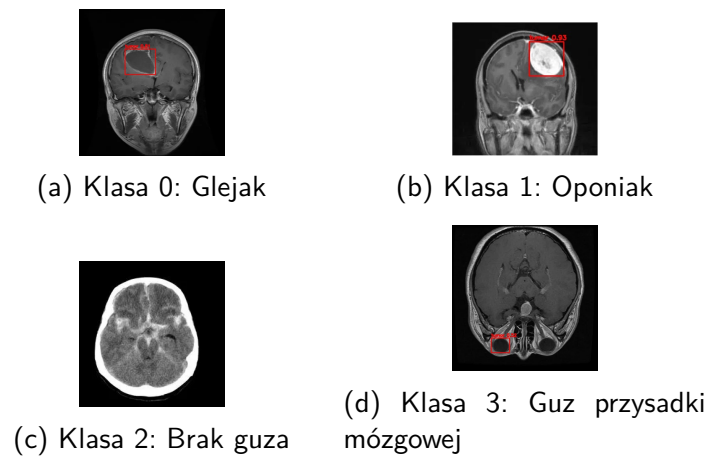


(f) VGG19

Rysunek 3.18: Wyniki działania sieci z ramkami wygenerowanymi przez bibliotekę OpenCV dla klasy 3: Guz przysadki mózgowej

Ramki wygenerowane przez sieci ResNet dla zdjęcia rezonansu magnetycznego z widocznym guzem przysadki mózgowej, są w poprawnym miejscu, ponieważ każda z nich zawiera w sobie guza. Poprzez dużą ilość zawartości mózgu niezmiennego chorobowo nie można powiedzieć, że są dokładne. Ramki wygenerowane przez sieci VGG są niedokładne i nie wskazują poprawnie obecnego na zdjęciu guza.

3.4.3 YOLOv8



Rysunek 3.19: Wyniki działania modelu YOLOv8 dla każdej z klas

Model YOLOv8 dobrze poradził sobie z zadaniem w przypadku klasy 0, czyli glejaka i klasy 1, czyli oponiaka. Poza tym w prawidłowy sposób nie znalazł ramek dla klasy 2, czyli brak guza. Zdjęcie klasy 3, czyli guza przysadki mózgowej, ze względu na obecność oczu okazało się największym wyzwaniem, ponieważ model zamiast guza zaznaczył jedno z oczu jako guz.

Prawdopodobieństwo wystąpienia klasy guz zwrócone przez model YOLOv8 na zdjęciu z glejakiem wynosi 81%, a prawdopodobieństwo wystąpienia tej klasy na zdjęciu z oponiakiem wynosi 93%. W przypadku braku wygenerowanej ramki przez model, model nie zwrócił prawdopodobieństwa, a błędnie oznaczone oko jako guz zwróciło prawdopodobieństwo 52%.

Rozdział 4

Wnioski

Rozdział ten powinien zawierać podsumowanie prezentujące usprawnienia i korzyści, jakie wnosi praca, uwagi co do napotkanych problemów i sposobu ich rozwiązania. Należy również nakreślić możliwości ulepszenia w przyszłości prezentowanej pracy.

Bibliografia

- [1] Programming languages for data science: 7 most in-demand right now, 18 November 2024. <https://www.index.dev/blog/programming-languages-for-data-science>. [str. 10]
- [2] Akmalbek Bobomirzaevich Abdusalomov, Mukhriddin Mukhiddinov, and Taeg Keun Whangbo. Brain tumor detection based on deep learning approaches and magnetic resonance imaging. *Cancers*, 15(16), 18 August 2023. <https://www.mdpi.com/2072-6694/15/16/4172>. [str. 14]
- [3] Rupesh Dulal and Rabin Dulal. Brain tumor identification using improved yolov8, 26 April 2025. <https://arxiv.org/pdf/2502.03746>. [str. 14]
- [4] Rozporządzenie parlamentu europejskiego i rady (ue) 2024/1689 z dnia 13 czerwca 2024 r. w sprawie ustanowienia zharmonizowanych przepisów dotyczących sztucznej inteligencji oraz zmiany rozporządzeń (we) nr 300/2008, (ue) nr 167/2013, (ue) nr 168/2013, (ue) 2018/858, (ue) 2018/1139 i (ue) 2019/2144 oraz dyrektyw 2014/90/ue, (ue) 2016/797 i (ue) 2020/1828 (akt w sprawie sztucznej inteligencji) (tekst mający znaczenie dla eog), 13 czerwca 2024. <https://eur-lex.europa.eu/legal-content/PL/TXT/?uri=CELEX:32024R1689>. [str. 7]
- [5] Tory O. Frizzell, Margit Glashutter, Careesa C. Liu, An Zeng, Dan Pan, Sujoy Ghosh Hajra, Ryan C.N. D'Arcy, and Xiaowei Song. Artificial intelligence in brain mri analysis of alzheimer's disease over the past 12 years: A systematic review. *Ageing Research Reviews*, 77:101614, 28 March 2022. <https://www.sciencedirect.com/science/article/pii/S1568163722000563>. [str. 13]
- [6] Shunmugavel Ganesh, Ramalingam Gomathi, and Suriyan Kannadhasan. Brain tumor segmentation and detection in mri using convolutional neural networks and vgg16. *Cancer Biomarkers*, 42(3):18758592241311184, 4 April 2025. PMID: 40183298 <https://journals.sagepub.com/doi/10.1177/18758592241311184>. [str. 14]

- [7] Best python libraries for machine learning, 11 July 2025. <https://www.geeksforgeeks.org/machine-learning/best-python-libraries-for-machine-learning/>. [str. 10]
- [8] Colaboratory - frequently asked questions. <https://research.google.com/colaboratory/faq.html>. [str. 11]
- [9] Conor Hale. Fda clears ai that predicts alzheimer's progression from an mri scan, 12 January 2024. <https://www.fiercebiotech.com/medtech/fda-clears-ai-predicts-alzheimers-progression-mri-scan>. [str. 13]
- [10] Ahmed Hamada. Br35h :: Brain tumor detection 2020. *IEEE Dataport*, 10 March 2025. <https://ieee-dataport.org/documents/br35h-brain-tumor-detection-2020-0>, <https://www.kaggle.com/datasets/ahmedhamada0/brain-tumor-detection>. [str. 24]
- [11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 27 - 30 June 2016. <https://arxiv.org/pdf/1512.03385>. [str. 21]
- [12] Thomas Hofweber and Rebecca L. Walker. Machine learning in health care: Ethical considerations tied to privacy, interpretability, and bias. *North Carolina Medical Journal*, 85(4), 11 July 2024. <https://ncmedicaljournal.com/article/120562-machine-learning-in-health-care-ethical-considerations-tied-to-privacy-interpretability-and-bi>
[str. 8, 9]
- [13] Katerina Hynkova. Ultimate guide to torchvision library in python, 22 August 2025. <https://deeptime.com/blog/ultimate-guide-to-torchvision-library-in-python>. [str. 12]
- [14] Zheshu Jia and Deyun Chen. Brain tumor identification and classification of mri images using deep learning techniques. *IEEE Access*, 13:123783–123792, 13 August 2020. <https://ieeexplore.ieee.org/document/9166580>. [str. 14]
- [15] Ankita Kadam, Sartaj Bhuvaji, and Sujit Deshpande. Brain tumor classification using deep learning algorithms. *Int. J. Res. Appl. Sci. Eng. Technol*, 9:417–426, December 2021. <https://www.ijraset.com/best-journal/brain-tumor-classification-using-deep-learning-algorithms>. [str. 15, 16]
- [16] Ayoosh Kathuria. Pytorch 101, understanding graphs, automatic differentiation and autograd, 25 September 2024. <https://www.digitalocean.com/community/tutorials/pytorch-101-understanding-graphs-and-automatic-differentiation>. [str. 11]

- [17] Sophie Magnet, 8 April 2025. <https://365datascience.com/career-advice/career-guides/data-scientist-job-outlook-2025/>. [str. 10]
- [18] Rafael Martínez-Del-Río-Ortega, Javier Civit-Masot, Francisco Luna-Perejón, and Manuel Domínguez-Morales. Brain tumor detection using magnetic resonance imaging and convolutional neural networks. *Big Data and Cognitive Computing*, 8(9), 21 September 2024. <https://www.mdpi.com/2504-2289/8/9/123>. [str. 14]
- [19] Chris McCormick. Colab gpus features pricing, 23 April 2024. <https://mccormickml.com/2024/04/23/colab-gpus-features-and-pricing/>. [str. 11]
- [20] Europejski ai act opublikowany, 12 lipca 2024. <https://www.gov.pl/web/cyfryzacja/europejski-ai-act-opublikowany>. [str. 6]
- [21] Riccardo Miotto, Fei Wang, Shuang Wang, Xiaoqian Jiang, and Joel T Dudley. Deep learning for healthcare: review, opportunities and challenges. *Briefings in Bioinformatics*, 19(6):1236–1246, 6 May 2017. <https://doi.org/10.1093/bib/bbx044>. [str. 8, 9]
- [22] How computational graphs are constructed in pytorch, 31 August 2021. <https://pytorch.org/blog/computational-graphs-constructed-in-pytorch/>. [str. 11]
- [23] Datasets. <https://docs.pytorch.org/vision/main/datasets.html>. [str. 12]
- [24] Getting started with transforms v2. https://docs.pytorch.org/vision/main/auto_examples/transforms/plot_transforms_getting_started.html. [str. 18]
- [25] Transforming images, videos, boxes and more. <https://docs.pytorch.org/vision/main/transforms.html>. [str. 12]
- [26] Transfer learning for computer vision tutorial, 27 January 2025. https://docs.pytorch.org/tutorials/beginner/transfer_learning_tutorial.html. [str. 12]
- [27] A Shahin. Fine-tuned resnet34 for efficient brain tumor classification. *Scientific Reports*, 15, 22 October 2025. <https://www.nature.com/articles/s41598-025-20872-3>. [str. 14]
- [28] Dyrbye LN Trockel M Tutty M Wang H sCarlasare LE Shanafelt TD, West CP and Sinsky C. Changes in burnout and satisfaction with work-life integration in physicians during the first 2 years of the covid-19 pandemic. *Mayo Clin Proc.*, 97(12):2248 – 2258, December 2022. <https://doi.org/10.1016/j.mayocp.2022.09.002>. [str. 6]

- [29] Afshin Shoeibi, Marjane Khodatars, Mahboobeh Jafari, Parisa Moridian, Mitra Rezaei, Roohallah Alizadehsani, Fahime Khozeimeh, Juan Manuel Gorriz, Jónathan Heras, Maryam Panahiazar, Saeid Nahavandi, and U. Rajendra Acharya. Applications of deep learning techniques for automated multiple sclerosis detection using magnetic resonance imaging: A review. *Computers in Biology and Medicine*, 136:104697, 9 August 2021. <https://arxiv.org/abs/2105.04881>. [str. 13]
- [30] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *CoRR*, abs/1409.1556, 4 September 2014. <https://arxiv.org/pdf/1409.1556>. [str. 19]
- [31] 2025 developer survey - technology, 2025. <https://survey.stackoverflow.co/2025/technology>. [str. 10]
- [32] Explore ultralytics yolov8, 10 January 2023. <https://docs.ultralytics.com/models/yolov8/>. [str. 25]
- [33] Occupational outlook handbook - data scientists, 2024. <https://www.bls.gov/ooh/math/data-scientists.htm>. [str. 10]
- [34] Jason Uwaeze, Ponnada A. Narayana, Arash Kamali, Vladimir Braverman, Michael A. Jacobs, and Alireza Akhbardeh. Automatic active lesion tracking in multiple sclerosis using unsupervised machine learning. *Diagnostics*, 14(6), 16 March 2024. <https://www.mdpi.com/2075-4418/14/6/632>. [str. 13]
- [35] Sukru Burc Eryilmaz Vartika Singh Michelle Lin Natalia Gimelshein Alban Desmaison Edward Yang Vinh Nguyen, Michael Carilli. Accelerating pytorch with cuda graphs, 26 October 2021. <https://pytorch.org/blog/accelerating-pytorch-with-cuda-graphs/>. [str. 12]

Spis rysunków

3.1	Wykres słupkowy z rozkładem danych	17
3.2	Przykładowe zdjęcia ze zbioru danych	17
3.3	Mapa cieplna sieci VGG16	20
3.4	Mapa cieplna sieci VGG16	20
3.5	Mapa cieplna sieci ResNet18	21
3.6	Mapa cieplna sieci ResNet50	22
3.7	Mapa cieplna sieci ResNet101	22
3.8	Mapa cieplna sieci ResNet152	23
3.9	Przykładowe zdjęcie ze zbioru danych Br35H: klasa z obecnym guzem mózgu . . .	24
3.10	Wykres słupkowy pokazujący ilość użytych i usuniętych danych	24
3.11	Wyniki działania sieci z Grad-CAM'em dla klasy 0: glejak	26
3.12	Wyniki działania sieci z Grad-CAM'em dla klasy 1: oponiak	27
3.13	Wyniki działania sieci z Grad-CAM'em dla klasy 2: Brak guza	28
3.14	Wyniki działania sieci z Grad-CAM'em dla klasy 3: Guz przysadki mózgowej	28
3.15	Wyniki działania sieci z ramkami wygenerowanymi przez bibliotekę OpenCV dla klasy 0: Glejak	29
3.16	Wyniki działania sieci z ramkami wygenerowanymi przez bibliotekę OpenCV dla klasy 1: Oponiak	30
3.17	Wyniki działania sieci z ramkami wygenerowanymi przez bibliotekę OpenCV dla klasy 2: Brak guza	30
3.18	Wyniki działania sieci z ramkami wygenerowanymi przez bibliotekę OpenCV dla klasy 3: Guz przysadki mózgowej	31
3.19	Wyniki działania modelu YOLOv8 dla każdej z klas	32

Spis tabel

3.1	Tabela porównawcza wyników treningu poszczególnych modeli	23
3.2	Tabela porównawcza wyników testu poszczególnych modeli	23

Załączniki

Jeśli jakieś elementy pracy są załączane.